

Deep Agents

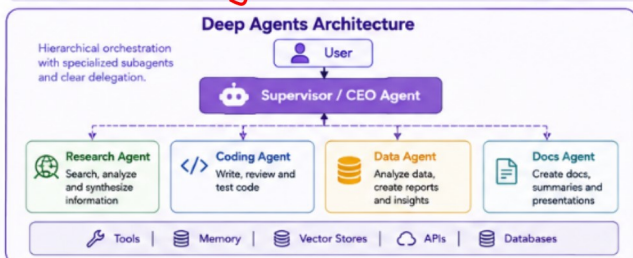


Deep Agents vs Claude Agent SDK

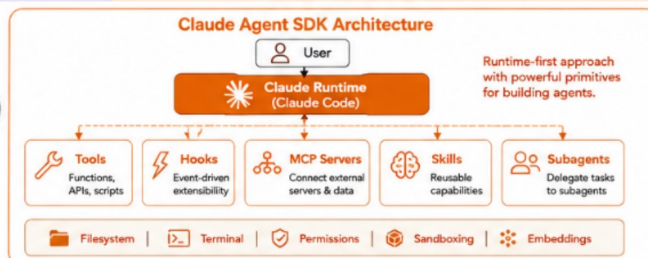
Two powerful approaches to building autonomous agents, different philosophies, same goal.



Feature	Deep Agents (LangChain)	Claude Agent SDK (Anthropic)
Primary Goal	Build long-running, multi-agent workflows	Extend Claude Code to build powerful agents
Foundation	LangChain + LangGraph	Claude Code infrastructure
Model Support	OpenAI, Anthropic, Gemini, Grok, local models	Claude models (Opus, Sonnet, Haiku)
Core Strength	Multi-agent orchestration & planning	Tool use, runtime, and developer productivity
Subagents	Native hierarchical subagents	Subagents & delegation via Claude runtime
Context Management	Explicit context isolation & quarantine	Managed by Claude Code runtime
Skills	Built-in skills system	Skills support through Claude ecosystem
Hooks / Events	Limited (via LangGraph workflows)	Powerful event-driven hooks
MCP Support	Yes	Yes
Multi-Agent Orchestration	Core design principle	Supported, not the primary focus
Human-in-the-loop	Via LangGraph workflows	Hooks, permissions & approvals
Observability	Native integration with LangSmith	External observability (logs, tracing, etc.)
Best Use Cases	Enterprise agents, RAG, research, analysis, autonomous workflows	Coding agents, CLI/IDE agents, DevOps, software engineering automation
Vendor Lock-in	Very Low	Higher (Claude ecosystem)



VS



Choose Deep Agents When

- You need multi-agent orchestration
- Long-running, complex workflows
- Enterprise systems & RAG applications
- Model flexibility is important

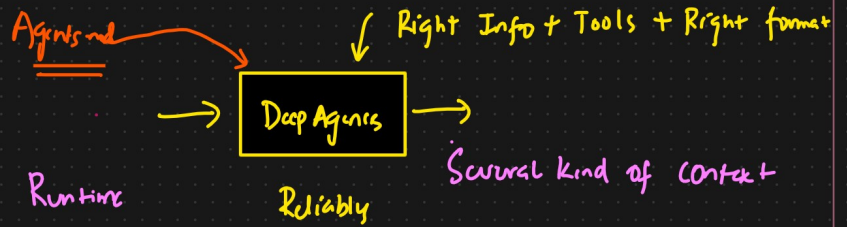
Bottom Line

Deep Agents is about **orchestrating a team of AI experts**.
Claude Agent SDK is about **extending a powerful AI colleague** that can code, act, and automate.

Choose Claude Agent SDK When

- You're building coding or CLI/IDE agents
- Developer productivity is the priority
- You want deep integration with Claude Code
- You need hooks, permissions & sandboxing

Context Engineering In Deep Agents



Claude Code → Claude.md

MEMORY IN DEEP AGENTS

Memory Files (AGENTS.md)

Memory files provide persistent context that is always loaded into the system prompt. Use memory for project conventions, user preferences, and critical guidelines that should apply to every conversation.

```
agent = create_deep_agent(  
    model="google_genai:gemini-3.5-flash",  
    memory=["/project/AGENTS.md", "~/.deepagents/preferences.md"],  
)
```

Memory is ALWAYS loaded into the system prompt. There is no progressive disclosure for memory.

Why Keep Memory Minimal?
Smaller memory → Fewer tokens → Lower cost
Faster responses → Less confusion for the model

HOW MEMORY IS USED (ARCHITECTURE)

USER QUERY
e.g., "Create a CRUD API"

SYSTEM PROMPT
Base instructions (behavior, tone, rules)

MEMORY FILES (ALWAYS LOADED)
/project/AGENTS.md
~/.deepagents/preferences.md

Injected into

FINAL PROMPT SENT TO LLM
System Prompt + Memory Files + User Query

LLM (Gemini 3.5 Flash)
Generates response using full context

MEMORY vs SKILLS

MEMORY	SKILLS
Always loaded	Loaded only when needed (Progressive Disclosure)
Used for global rules, preferences, architecture, conventions	Used for detailed workflows, domain knowledge, complex procedures
Small, concise, high-level context	Large, detailed, situational context
Example: Coding standards, Backend architecture, Security rules, Preferences	Example: Kubernetes, AWS, LangGraph, FastAPI, Neo4j

Example

User: "Deploy FastAPI app to EKS"

Memory provides:

- Cloud: AWS
- Orchestration: Kubernetes
- Framework: FastAPI
- Always available

Skills loaded:

- AWS Skill
- Kubernetes Skill
- (Not React Skill)
- Loaded on demand

EXAMPLE: AGENTS.md (MEMORY FILE)

```
1 # KrishAI Architecture  
2  
3 Backend:  
4 - FastAPI  
5  
6 Database:  
7 - PostgreSQL  
8  
9 Cache: Redis  
10 Vector DB: Qdrant  
11 Cloud: AWS (Kubernetes)
```

What the agent knows automatically:

- Use FastAPI
- Use PostgreSQL
- Use Qdrant for vectors
- Use Redis for cache
- Deploy on Kubernetes
- Cloud: AWS

Now when the user asks "Build a RAG pipeline", the agent already knows your entire backend stack.

HOW MEMORY CONNECTS TO BACKENDS

Memory files store your backend architecture so the agent always uses the right components.

PostgreSQL (Database) Redis (Cache) Elasticsearch (Search) Neo4j (Graph DB) AWS S3 (Storage)

AGENT

Benefits

- Consistent architecture usage
- Fewer mistakes
- Follows company standards
- Secure by default

BEST PRACTICES FOR MEMORY

- Keep it minimal and focused
Only essential rules, architecture, and preferences.
- Use bullets and structure
Easy for the model to read and follow.
- Include critical security guidelines
Secrets handling, auth, data protection rules.
- Update as your system evolves
Keep AGENTS.md versioned with your project.
- Use multiple memory files if needed
Separate global rules, project rules, and user prefs.

Key Takeaway: Memory (AGENTS.md) = Always-on, persistent, high-level context. Keep it small.
Skills = On-demand, detailed, domain-specific knowledge. Use for complex workflows.

Together, they make your Deep Agent consistent, efficient, and production-ready.

1) Input Context → What goes into agents prompt at startup

[System prompt, memory, skills]

Scope

Static, applied each run

2) Runtime Context ✓

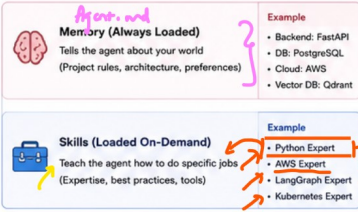
3) Context Compression ✓

4) Context Isolation ✓

SKILLS IN DEEP AGENTS

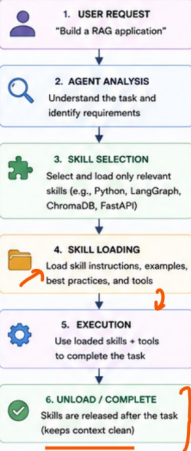
On-demand expertise with progressive disclosure

Skills are specialized capability modules that the agent loads only when they are relevant to the user's request.



Key Idea: Don't load everything all the time. Load the right skill, at the right time, for the right task.

HOW SKILLS WORK (LIFECYCLE)

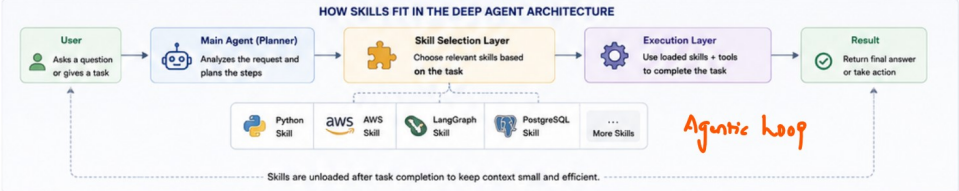
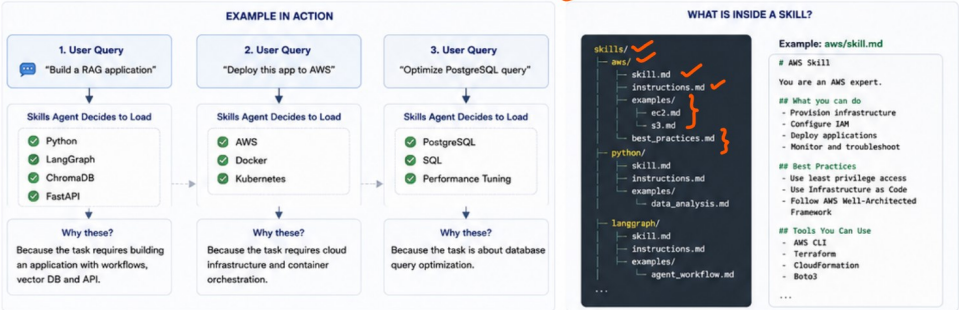


MEMORY vs SKILLS

MEMORY (Always Loaded)	SKILLS (Loaded On-Demand)
Always injected into system prompt	Loaded only when relevant
Global rules, preferences, architecture, conventions	Detailed workflows, domain knowledge, step-by-step guidance
Small, high-level, always relevant	Large, deep, situational
Example: Coding standards, backend architecture, security rules	Example: Kubernetes deployment, LangGraph workflows, AWS setup
No progressive disclosure	Progressive disclosure (load when needed)

EXAMPLE SCENARIO

User Request: "Create a Graph RAG system"	User Request: "Deploy it to AWS"
Skills Loaded: Python, LangGraph, Neo4j, ChromaDB	Skills Loaded: AWS, Docker, Kubernetes



ONE LINE TAKEAWAY Memory gives the agent context about your world. Skills give the agent expertise to get the job done.

Deep Agents → To do that
↳ ↑



{Virtual file system}

↓

Tools

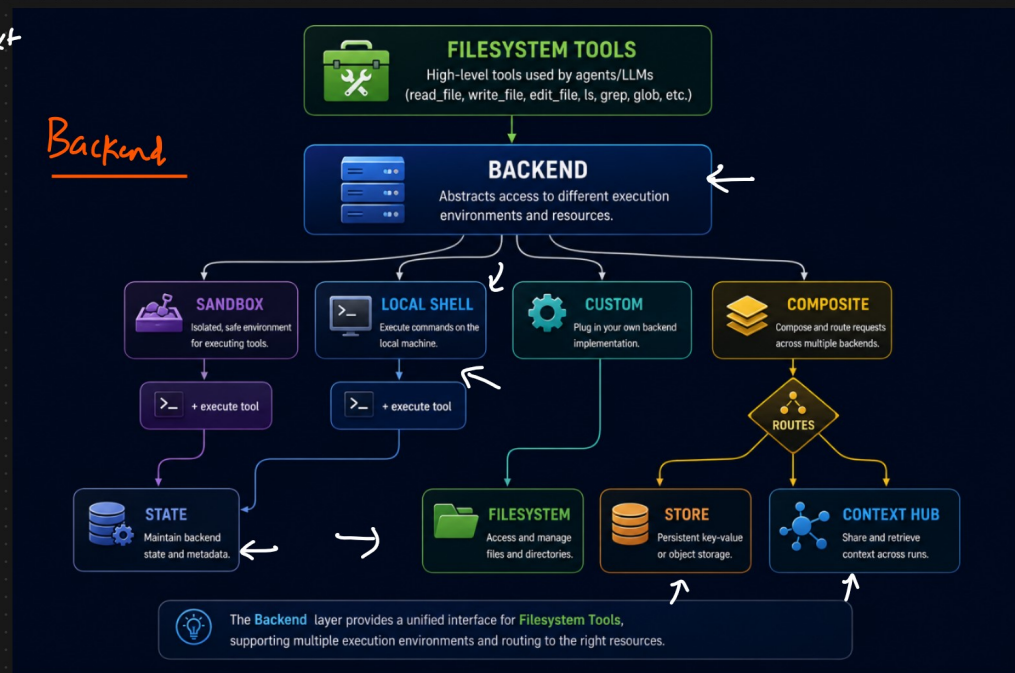
- 1) Write files
- 2) Read files
- 3) Edit files

Saved

2

Langgraph State

→ Thread





SUBAGENTS IN DEEP AGENTS (LANGCHAIN)

Delegate. Isolate. Execute.

Subagents are specialized AI agents that the main agent delegates tasks to. Each subagent has its own context, tools, memory and instructions to perform a focused job and return results.

WHY USE SUBAGENTS?

- ✓ Break complex tasks into manageable parts
- ✓ Run tasks in parallel for speed
- ✓ Isolate context to reduce noise and token usage
- ✓ Use the right tools and skills for each job
- ✓ Improve reliability and maintainability

SUBAGENT VS MAIN AGENT



MAIN AGENT

- Understands the overall goal
- Plans and decomposes tasks
- Delegates to subagents
- Aggregates all results
- Returns final answer



SUBAGENTS

- Focused on a specific task
- Have isolated context
- Use specialized tools/skills
- Return structured results
- Do not decide next steps

KEY TAKEAWAY



Subagents turn one generalist agent into a team of specialists.



Delegate the right task to the right expert.

CONTEXT ISOLATION (IMPORTANT!)



Main Agent

Global memory & high-level context
Knows the big picture



Subagents

Local memory & task-specific context
Know only what they need

- ✓ Isolation reduces noise, saves tokens and improves accuracy.

HOW SUBAGENTS WORK (HIGH LEVEL FLOW)



1. USER REQUEST

Example: "Build a research report on Agentic AI"



2. MAIN AGENT (ORCHESTRATOR)

Understands the goal, creates a plan and decides which subagents are needed.



3. DELEGATE TO SUBAGENTS

Main agent assigns each task to the right subagent based on expertise.



RESEARCH AGENT

- Web search
- Read sources
- Extract insights



WRITING AGENT

- Write sections
- Structure content
- Improve clarity



DATA AGENT

- Analyze data
- Create charts
- Summarize stats



CRITIQUE AGENT

- Review quality
- Fact check
- Suggest fixes



4. SUBAGENTS EXECUTE (IN PARALLEL)

Each subagent works independently using its own tools, context and memory.



5. RESULTS RETURNED

Subagents return structured results to the main agent.



6. MAIN AGENT AGGREGATES

Combines, validates and synthesizes all results.



7. FINAL ANSWER TO USER

Main agent delivers the complete, high-quality response.

SUBAGENT CAPABILITIES



Independent Context

Each subagent has isolated context and instructions.



Specialized Tools & Skills

Uses only the tools and skills it needs.



Parallel Execution

Subagents can work in parallel to save time.



Failure Isolation

If one subagent fails, others can still succeed.



Result Hand-off

Subagents return structured results to the main agent.

EXAMPLE: RESEARCH REPORT WORKFLOW

- 1 User asks: "Build a research report on Agentic AI"
- 2 Main agent creates a plan and delegates:
 - Research Agent → gather information
 - Data Agent → analyze market data
 - Writing Agent → draft the report
 - Critique Agent → review and improve
- 3 Subagents work in parallel
- 4 Results returned to main agent
- 5 Main agent synthesizes final report and returns to user

BEST PRACTICES

- ✓ Keep subagents focused and single-purpose
- ✓ Give clear instructions and expected output format
- ✓ Use context isolation to reduce token usage and confusion
- ✓ Run independent tasks in parallel whenever possible
- ✓ Design robust hand-off formats (JSON, Markdown, etc.)
- ✓ Log and monitor each subagent for debugging and observability

WHEN TO USE SUBAGENTS?



Complex, multi-step tasks



Tasks that can run in parallel



Different domains or expertise



Large context or data



Quality check or validation

- ✓ If a task needs multiple experts in real life, you need subagents in AI.